Text Sentiment Classification For Birch

Author
KHAN, Zeeshan Alam
PA3 Group 65
Chalmers University of Technology
zeeshank@chalmers.se

1. Introduction

This report documents the development of a sentiment classification system, designed to automatically determine whether a given piece of text expresses a positive, negative, or neutral sentiment. The system was trained and evaluated on a dataset of tweets.

The project covers the full development pipeline, starting from data annotation and exploration, through data cleaning, feature extraction, model training and hyperparameter tuning, and ending with a final evaluation on a held-out test set. A key aspect of this project is the comparison between two versions of the training data: one labelled through crowdsourcing and one verified by more trusted sources. This allows us to directly measure how annotation quality affects model performance, which is a practically important question for any organisation considering deploying a machine learning system.

2. Annotated Datasets

In this project, we are working with three primary datasets to evaluate and build a sentiment classifier. All datasets follow a consistent schema with two fields: “sentiment” (labeled as positive, negative, or neutral) and “text” (containing the tweet content of which the sentiment was analyzed).

The first dataset, *crowdsourced_train.csv*, contains sentiment labels collected through crowdsourcing, meaning that the tweets were annotated by non-experts (students) rather than domain specialists. The second dataset, *gold_train.csv*, includes the same tweets but with labels that have been verified by more reliable sources. Having both versions of the same data allows for a direct comparison between crowdsourced annotations and trusted gold labels. In this project, both datasets are used for training and analysis. The third dataset, *test.csv*, is a separate held-out dataset used only for final evaluation. It contains tweets that do not appear in the training data, ensuring that the evaluation reflects the model’s ability to generalise rather than memorise.

3. Data Exploration and Cleaning

3.1. Label Inconsistencies In Crowdsourced Dataset

We explored the crowdsourced dataset (*crowdsourced_train.csv*) to identify any issues before training the model. During this step, we found several inconsistencies in the “sentiment” column. These included differences in capitalization (e.g., Positive vs positive), spelling mistakes (such as positve, negayive), and extra spaces or malformed entries.

3.2. Data Cleaning For Crowdsourced Dataset

To fix the label inconsistencies found in the dataset, we applied a few cleaning steps. First, all sentiment labels were converted to lowercase and had any extra whitespace removed. We then built a manual mapping to correct spelling mistakes and odd variations, replacing entries like positve, negayive and neutal with their correct forms.

After cleaning, we checked the unique values in the sentiment column to confirm the result. Only the three expected labels remained: negative, neutral and positive. This gave us a consistent dataset to work with for the rest of the project.

3.3. Crowdsourced Dataset Split

After cleaning the labels, we looked at how the classes were distributed across the dataset. As shown in **Table 1.1**, the dataset is not perfectly balanced. Neutral is the largest class with 5,079 examples (47.57%), followed by positive with 3,219 examples (30.15%) and negative with 2,378 examples (22.27%).

Sentiment Label	Count	Percentage
neutral	5079	47.57%
positive	3219	30.15%
negative	2378	22.27%

Table 1.1: Crowdsourced dataset label distribution

3.4. Gold Dataset Split

We then loaded the gold dataset (gold_train.csv). Unlike the crowdsourced data, checking the unique labels before cleaning already showed only three valid values: neutral, positive and negative. No spelling mistakes or formatting issues were present, which reflects the higher quality of the gold annotations.

The label distribution, shown in **Table 1.2**, followed a similar pattern to the crowdsourced set. Neutral was again the dominant class with 5,364 examples (50.24%), followed by positive with 3,652 (34.21%) and negative with 1,660 (15.55%). Despite the cleaner labels, we still applied the same cleaning steps (lowercasing and whitespace removal) for consistency, and confirmed afterwards that only the three expected labels remained.

Sentiment Label	Count	Percentage
neutral	5364	50.24%
positive	3652	34.21%
negative	1660	15.55%

Table 1.2: Gold dataset label distribution

3.5. Comparing Crowdsourced and Gold Labels

To understand how well the crowdsourced annotations align with the gold standard, we compared the two label distributions and calculated Cohen's Kappa score, which accounts for agreement that could occur by chance. A score of 1 means perfect agreement, 0 means no better than chance, and negative values indicate systematic disagreement. The result was a Kappa score of 0.4463, which falls in the moderate agreement range. This means the two annotation sources agree on many cases, but a considerable number of disagreements remain. For a model trained on crowdsourced labels, this introduces a degree of label noise that may limit how well it generalises to the gold-labelled test set.

4. Model Selection, Hyperparameter Tuning and Evaluation

4.1. Text Vectorization

To train our classifier, the raw tweet text first needs to be converted into numerical features that a machine learning model can work with. We used TF-IDF (Term Frequency-Inverse Document Frequency) vectorization for this, which represents each tweet as a vector of word weights. Words that appear often in a specific tweet but rarely across the whole dataset get a higher weight, making them more useful for distinguishing between classes. Common words like "the" or "is" naturally receive low weights since they appear everywhere and carry little sentiment signals.

We applied the same TF-IDF vectorizer to both the crowdsourced and gold datasets. Both datasets originally had a shape of (10,676, 2), representing 10,676 tweets across two columns: sentiment and text. After vectorization, the shape expanded to (10,676, 22,633), where each tweet is now represented by 22,633 numerical features, one for each unique word found across the entire dataset.

4.2. Model Selection and Hyperparameter Tuning

To identify the best classifier for this task, we trained and evaluated three models: Logistic Regression, Multinomial Naive Bayes, and Random Forest. Rather than manually testing each combination of settings, we used GridSearchCV with 5-fold cross-validation to systematically search for the best hyperparameters for each model. The same fold splits were used across all experiments by initialising KFold once with a fixed random seed, ensuring a fair comparison between models.

For Logistic Regression, we tested regularisation strength C across values 0.1, 1 and 10, and two solvers: liblinear and lbfgs. For Naive Bayes, the smoothing parameter alpha was tested at 0.1, 0.5 and 1.0. For Random Forest, we varied the number of trees between 100 and 200, and the maximum tree depth across 10, 20 and no limit. This process was run separately on both the crowdsourced and gold training sets. As shown in **Table 1.3 and Table 1.4**, Logistic Regression came out on top in both cases, achieving a cross-validation accuracy of 0.5738 on crowdsourced labels and 0.6383 on gold labels, with best parameters C=1 and solver=lbfgs.

--- Model Performance Summary (Crowdsourced Labels) ---

	Model	Best Score	Best Params
0	Logistic Regression	0.573809	{'C': 1, 'solver': 'lbfgs'}
2	Random Forest	0.562289	{'max_depth': None, 'n_estimators': 200}
1	Naive Bayes	0.547113	{'alpha': 0.5}

Table 1.3: Crowdsourced dataset performance

--- Model Performance Summary (Gold Labels) ---

	Model	Best Score	Best Params
0	Logistic Regression	0.638348	{'C': 1, 'solver': 'lbfgs'}
2	Random Forest	0.623642	{'max_depth': None, 'n_estimators': 200}
1	Naive Bayes	0.606313	{'alpha': 0.5}

Table 1.4: Gold dataset performance

4.3. Final Model Evaluation on Test Set

The best model, Logistic Regression, was retrained on the full training set and evaluated on the held-out test.csv. The crowdsourced-trained model achieved a test accuracy of 61.95%, while the gold-trained model reached 71.45%. a notable improvement that reflects the higher quality of gold labels during training. Results are shown on **Table 1.5**.

Final Model Performance on test.csv:

	Training Source	Test Accuracy
0	Crowdsourced	0.619479
1	Gold	0.714518

Table 1.5: Performance of best training model on the test set

4.4. Confusion Matrix Analysis

To better understand where the models were succeeding and failing, confusion matrices were generated for both the crowdsourced-trained and gold-trained models on the test set, shown in **Figure 1**. In the crowdsourced-trained model, the neutral class was classified reasonably well with 2,050 correct predictions, but positive tweets were heavily misclassified as neutral, with 896 positive tweets incorrectly predicted as neutral against only 877 correct positive predictions. The negative class also struggled, with 518 negative tweets wrongly predicted as neutral. This suggests the crowdsourced model had a strong bias towards predicting neutral, likely a consequence of the noisier training labels.

The gold-trained model showed a clearer improvement across all classes. Neutral remained the strongest class with 2,259 correct predictions and far fewer errors. Positive improved significantly with 1,237 correct predictions, though 586 positive tweets were still misclassified as neutral. Negative remained the hardest class for both models, with the gold model correctly identifying 451 negative tweets but still misclassifying 540 as neutral.

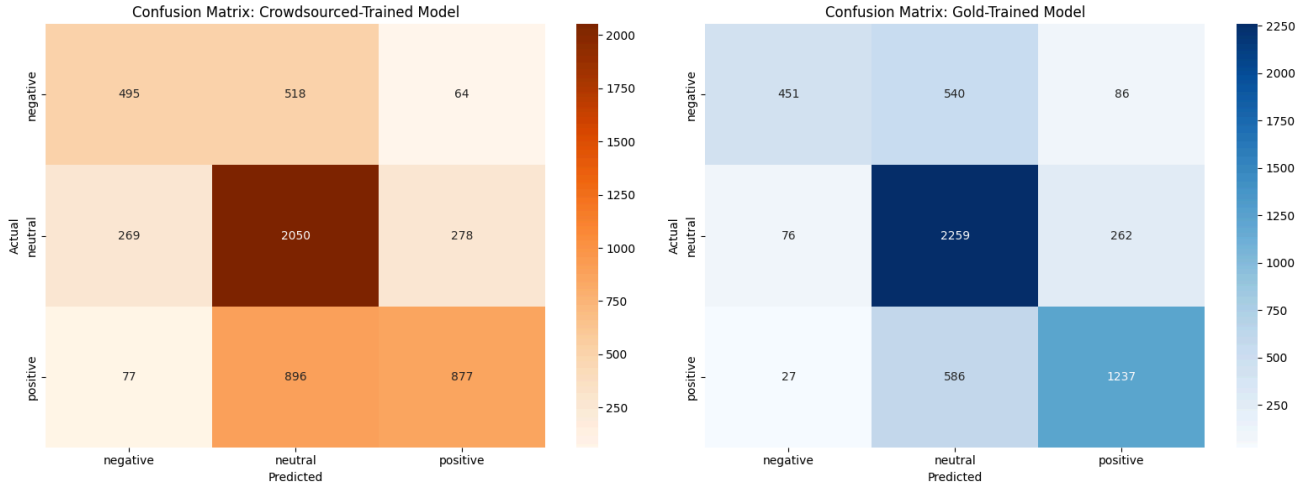

Figure 1: Confusion matrices for crowdsourced and gold training model performance on test data

5. Analysis

The results of this project demonstrate a clear and consistent finding: annotation quality has a measurable and significant impact on classifier performance. The gold-trained Logistic Regression model achieved a test accuracy of 71.45%, compared to 61.95% for the crowdsourced-trained model, a gap of nearly 10 percentage points on otherwise identical data and hyperparameters. This difference can be directly attributed to the label noise present in the crowdsourced dataset, as reflected by Cohen's Kappa score of 0.4463. When annotators disagree on roughly a quarter of labels, the model learns a noisier decision boundary, which compounds into weaker generalisation.

A key limitation of the current system is that TF-IDF treats each word independently and ignores word order, context, and negation. A tweet like "not bad at all" shares vocabulary with both positive and negative examples and offers no structural cues to a bag-of-words model. This likely contributes to persistent misclassifications, particularly for the negative class, where sentiment is often expressed indirectly.

Logistic Regression consistently outperformed both Naive Bayes and Random Forest across both training sets. This is plausible: logistic regression is well-suited to high-dimensional sparse feature spaces like TF-IDF, and its linear decision boundary is often sufficient when features are informative. Naive Bayes

performed comparably but makes a strong conditional independence assumption that is unlikely to hold for natural language. Random Forest tends to require denser, lower-dimensional input to perform well and is generally less competitive on raw bag-of-words representations.

6. Recommendation and Conclusion

Looking ahead, several improvements could be made if the system continues to rely on crowdsourced data. Before annotation begins, clear and detailed guidelines should be provided to annotators, including concrete examples of each sentiment class and how to handle borderline cases such as sarcasm, mixed opinions, and factual statements, as these are the primary sources of disagreement. Running a small pilot round before full-scale annotation is also advisable; distributing a batch of pre-labelled examples to annotators first and checking their responses against a trusted reference quickly identifies misunderstandings before they spread across the full dataset. Where budget allows, having multiple annotators label the same item and resolving disagreements through majority vote reduces the impact of individual errors significantly. Agreement scores such as Cohen's Kappa should be computed regularly throughout the annotation process rather than only at the end, so that problems are caught and corrected early. On the modelling side, addressing the class imbalance through oversampling or class weighting, and eventually replacing TF-IDF with a contextual pretrained model such as BERTweet would all be expected to improve performance meaningfully. Together, these steps would reduce the label noise that was the single largest limiting factor in this project and bring the crowdsourced model closer in performance to one trained on gold-standard annotations.

7. Conclusion

This project successfully developed and evaluated a sentiment classification pipeline covering the complete development lifecycle, from data cleaning and annotation quality analysis through feature extraction, model selection, hyperparameter tuning, and final test evaluation. Logistic Regression with TF-IDF vectorization proved to be the strongest approach among the three models tested, and the comparison between crowdsourced and gold training data provided a direct and practical demonstration of how annotation quality affects deployed system performance. The 10-point accuracy gap between the two conditions is not attributable to any algorithmic difference but solely to the quality of the training signal, reinforcing that clean and consistent labels are as important as model choice. For any organisation considering a similar system, investing in annotation quality, whether through clearer guidelines, pilot rounds, or multi-annotator agreement checks, is likely to yield greater returns than architectural complexity alone.